

Fiche de revision L12 v2

Shell Unix + Virtualisation

Cours : Systemes d'exploitation de type Unix

Evaluation : Lecon 12 - 06.05.2026

Format : cours structure, commandes, notions et pieges integres

Sommaire

PARTIE 1 — SHELL

1. Unix, shell et aide
2. Commandes de base, chemins et fichiers
3. Ordre de recherche des commandes, PATH et historique
4. Expansions, quoting et globbing
5. Redirections, pipes et code de sortie
6. Filtres de texte
7. Recherche avec `find`
8. Permissions et droits d'accès
9. Processus et jobs
10. Variables, environnement et alias
11. Scripts Bash robustes
12. Utilisateurs et groupes

PARTIE 2 — VIRTUALISATION

1. Concepts
2. Installation, vérification et accès libvirt
3. VM de référence pour l'évaluation
4. Administration des VM avec `virsh`
5. Création de VM avec `virt-install`
6. Stockage, formats, pools et volumes
7. Clonage et post-installation
8. Console série
9. LVM, extension et snapshots
10. Réseaux virtuels libvirt
11. Topologie vm1, vm2, vm3 et passerelle
12. Configuration réseau dans une VM
13. SSH, clés, jump host et tunnel
14. Post-installation et contrôles finaux

ANNEXES

- Cheatsheet 1 page — commandes probables au test
- Pièges fréquents au test

PARTIE 1 - SHELL

1. Unix, shell et aide

Role du shell

Le shell est l'interpreteur de commandes. Il lit une ligne, effectue les expansions, cherche la commande, lance un processus, puis renvoie un code de sortie. Une interface graphique facilite les actions simples; le shell permet surtout d'automatiser, de combiner et de reproduire des operations precises.

Aide et documentation

Commande	Utilite
<code>man passwd</code>	manuel d'une commande
<code>man man</code>	manuel du manuel
<code>apropos password</code>	recherche dans les descriptions de manuels
<code>help</code>	liste des commandes internes Bash
<code>help cd</code>	aide sur une commande interne
<code>type cd</code> / <code>type ls</code>	indique comment le shell resout un nom
<code>which ls</code> / <code>command -v ls</code>	chemin de l'executable trouve

Piege : `cd`, `alias`, `export`, `read`, `source` sont des commandes internes. Elles doivent agir sur le shell courant; un executable externe ne pourrait pas changer le repertoire ou les variables du parent.

2. Commandes de base, chemins et fichiers

Identite et systeme

Commande	Effet
<code>whoami</code> / <code>logname</code>	utilisateur courant / login
<code>id</code> / <code>id user</code>	UID, GID, groupes secondaires
<code>who</code> / <code>w</code> / <code>users</code>	utilisateurs connectes
<code>tty</code>	terminal courant
<code>hostname</code>	nom de la machine
<code>date</code> / <code>cal</code>	date / calendrier
<code>pwd</code>	repertoire courant
<code>df -h</code>	espace disque
<code>free -h</code>	memoire
<code>ps</code> , <code>ps -ef</code> , <code>ps -axf</code>	processus
<code>clear</code>	nettoyer l'ecran

Chemins absolus et relatifs

Un chemin absolu commence par `/` et ne depend pas du dossier courant : `/etc/passwd`. Un chemin relatif depend de `pwd` : `./script.sh`, `../notes`, `tmp/file`.

```
pwd
cd                # retourne dans $HOME
cd ~user          # home d'un autre utilisateur
cd -              # repertoire precedent
cd ..            # parent
```

Listage et manipulation

```
ls
ls -a             # inclut les fichiers caches
ls -l            # format long
ls -lh           # tailles lisibles
ls -li           # ajoute l'inode

mkdir test
mkdir -p a/b/c
touch fichier.txt

cp source dest
cp -r dossier dest
mv ancien nouveau
rm fichier
rm -r dossier
rm -rf dossier    # dangereux

file fichier
file -L lien
```

Liens physiques et symboliques

Type	Commande	Idee
Lien physique	<code>ln cible lien</code>	deux noms pointent vers le meme inode
Lien symbolique	<code>ln -s cible lien</code>	fichier special contenant un chemin

Un lien physique continue de fonctionner si le nom original est supprime, tant qu'il reste au moins un lien vers l'inode. Un lien symbolique casse si sa cible n'existe plus.

```
echo test > a.txt
ln a.txt hard.txt
ln -s a.txt soft.txt
ls -li a.txt hard.txt soft.txt
```

3. Ordre de recherche des commandes, PATH et historique

Ordre de resolution

Quand on tape `nom`, Bash cherche en gros dans cet ordre : alias, mots reserves/fonctions, commandes internes, executables trouves dans `$PATH`.

```
type ls
type cd
type monScript
command -v monScript
```

PATH

`$PATH` est une liste de dossiers separes par `:`. Le shell y cherche les executables lorsqu'on ne donne pas de chemin.

```
echo "$PATH"
mkdir -p "$HOME/bin"
export PATH="$PATH:$HOME/bin"
```

Pour rendre ce changement permanent, ajouter la ligne dans `~/.bashrc` ou `~/.profile`, puis recharger :

```
source ~/.bashrc
. ~/.bashrc
```

Piege : lancer `script.sh` ne marche pas si le repertoire courant n'est pas dans `$PATH`. Utiliser `./script.sh` ou placer le script dans un dossier du `$PATH`.

Historique

```
history
!42      # relance la commande numero 42
!!       # relance la commande precedente
```

L'historique aide a retrouver des commandes, mais pendant un test il faut comprendre chaque option reutilisee.

4. Expansions, quoting et globbing

Ordre mental important

Avant d'appeler le programme, le shell transforme la ligne : expansion de variables, substitution de commandes, expansion d'accolades, word splitting, globbing, puis redirections.

```
echo "$HOME"
echo "date=$(date)"
echo {1..5}
```

Globbing

Motif	Sens
<code>*</code>	n'importe quelle chaine
<code>?</code>	exactement un caractere
<code>[abc]</code>	un caractere parmi <code>a</code> , <code>b</code> , <code>c</code>
<code>[a-z]</code>	un caractere dans l'intervalle
<code>[!abc]</code>	un caractere sauf <code>a</code> , <code>b</code> , <code>c</code>

```
touch data_{01..15}.txt
cp /usr/include/{a,c,m}*.h ~/trav1
```

Quotes

Syntaxe	Effet
'texte \$USER'	rien n'est interprete
"texte \$USER"	variables et substitutions restent actives
*	protege un seul caractere special

Toujours proteger les variables dans les scripts : "\$var". Sans quotes, le shell fait du word splitting puis du globbing.

```
fichier="mon fichier.txt"
rm $fichier      # deux arguments: mon et fichier.txt
rm "$fichier"    # un seul argument

var="*"
echo $var        # peut lister les fichiers courants
echo "$var"      # affiche *
```

Piege classique avec find

```
find /usr/include -name *.h      # faux si le shell developpe *.h
find /usr/include -name '*.h'    # correct
```

Avec `find -name`, proteger le motif. C'est `find`, pas le shell, qui doit interpreter `*.h`.

5. Redirections, pipes et code de sortie

Descripteurs standards

FD	Nom	Role
0	stdin	entree standard
1	stdout	sortie normale
2	stderr	erreurs

```
cmd > out.txt      # stdout, ecrase
cmd >> out.txt     # stdout, ajoute
cmd < in.txt       # stdin depuis fichier
cmd 2> err.txt     # stderr
cmd > out.txt 2>&1  # stdout + stderr dans le meme fichier
cmd &> out.txt     # raccourci Bash
find / 2>/dev/null # masquer les erreurs
```

Dans un script, envoyer une erreur sur `stderr` :

```
echo "Erreur: argument manquant" >&2
exit 1
```

Pipes et tee

```
cut -d: -f1 /etc/passwd | sort | uniq
ls | wc -l
ls -l | tee listing.log | grep '\.txt$'
```

Le pipe transmet le stdout de la commande de gauche au stdin de la commande de droite. `tee` duplique la sortie : affichage + fichier + suite du pipe.

Chainage logique

Operateur	Effet
;	execute toujours la suite
&&	execute la suite si succes
	execute la suite si echec

```
make && ./run
ping -c1 host || echo "K0"
cmd; echo "$?"
```

`$?` vaut `0` en cas de succes, non-zero en cas d'erreur.

6. Filtres de texte

Visualisation

```
cat fichier
less fichier
head -n 5 fichier
tail -n 20 fichier
tail -f /var/log/syslog
```

grep

```
grep motif fichier
grep -i motif fichier      # ignore la casse
grep -v motif fichier      # inverse
grep -n motif fichier      # numeros de ligne
grep -r motif dossier/     # recursif
grep -E '^[a-z]+$' fichier # regex etendue
grep '\.txt$' listing.txt  # point litteral + fin de ligne
grep -F '.txt' listing.txt # chaine fixe, pas regex
```

Piege : dans une regex, `.` signifie "n'importe quel caractere". Pour chercher un vrai point, utiliser `\.` ou `grep -F`.

cut, tr, sort, uniq, wc

```
cut -d: -f1 /etc/passwd
cut -d: -f1,3 /etc/passwd
cut -c1-10 fichier

tr 'a-z' 'A-Z'
tr -s ' '
tr -d ' '

sort fichier
sort -n nombres.txt
sort -r fichier
sort fichier | uniq
sort fichier | uniq -c

wc -l fichier
wc -w fichier
wc -c fichier
```

awk

```
awk '{print $1}' fichier
awk -F: '{print $1, $3}' /etc/passwd
awk -F: '$3 >= 1000 && $3 <= 1999 {print $1}' /etc/passwd
awk '{sum += $1} END {print sum}' nombres.txt
```

awk est souvent plus fiable que des enchainements trop fragiles de **cut** quand les colonnes sont numeriques ou conditionnelles.

diff et cmp

```
diff a.txt b.txt
diff -y a.txt b.txt
cmp a.bin b.bin
```

7. Recherche avec find

```
find . -name '*.txt'
find . -iname '*.TXT'
find . -type f
find . -type d
find . -type l
find . -size +1M
find . -mtime -7
find . -mtime +30
find . -user lucien
find . -group users
```

Permissions avec find

```
find . -perm 644
find . -perm -111
find . -perm -u=x,g=x,o=x
find . -perm /222
```

Forme	Sens
-perm 644	exactement ces droits
-perm -111	au moins ces bits
-perm /222	au moins un de ces bits

Actions

```
find . -name '*.tmp' -delete
find . -name '*.sh' -exec chmod +x {} \;
find . -type f -exec grep -l motif {} \;
find /etc -name '*.conf' -exec cp {} ~/backup_conf/ \; 2>/dev/null
```

8. Permissions et droits d'accès

Lire un **ls -l**

```
-rw-r--r-- 1 lucien users 1234 Mar 18 12:00 fichier.txt
|  |  |  |
| u  g  o
type
```


Type	Sens
-	fichier ordinaire
d	repertoire
l	lien symbolique
b / c	peripherique bloc / caractere

Octal

Octal	Droits
0	---
1	--x
2	-w-
3	-wx
4	r--
5	r-x
6	rw-
7	rwX

```

chmod u+x script.sh
chmod g-w fichier
chmod a=r fichier
chmod u=rwx,g=rx,o= dossier

chmod 644 fichier
chmod 755 dossier
chmod 600 ~/.ssh/id_rsa
chmod 700 ~/.ssh

```

Proprietaire et groupe

```

sudo chown lucien fichier
sudo chown lucien:users fichier
sudo chown -R lucien dossier/
sudo chgrp users fichier

```

Droits sur repertoires

Sur un repertoire, **x** signifie traverser/resoudre les noms, **r** signifie lister les noms, **w** signifie creer/supprimer/renommer des entrees.

Droits dossier	Effet
rwX	lister, traverser, creer/supprimer
r-x	lister et traverser, pas creer
--x	accéder a un nom connu, sans lister
rw-	presque inutilisable car pas de traverser
-wX	creer/supprimer si on connait le chemin, sans lister

Point cle : supprimer un fichier depend surtout du droit **w+X** sur le repertoire parent, pas du droit **w** sur le fichier lui-meme.

Droits minimums

Operation	Dossier source	Fichier source	Dossier destination
Lire <code>src/foo</code>	<code>--x</code>	<code>r--</code>	-
Copier <code>src/foo</code> vers <code>dst/</code>	<code>--x</code>	<code>r--</code>	<code>-wx</code>
Deplacer <code>src/foo</code> vers <code>dst/</code>	<code>-wx</code>	souvent aucun si meme FS	<code>-wx</code>
Supprimer <code>src/foo</code>	<code>-wx</code>	pas necessaire	-
Lister <code>src/</code>	<code>r--</code>	-	-
<code>cd src/</code>	<code>--x</code>	-	-

Methode pour exercices de permissions type activite 004

Dans ces exercices, il faut traduire une phrase en droits sur **deux niveaux differents** :

1. le droit sur le **dossier**, qui controle les noms dans le repertoire ;
2. le droit sur le **fichier**, qui controle le contenu du fichier.

La grosse idee : le dossier donne le droit d'arriver jusqu'au fichier, le fichier donne le droit de lire ou modifier son contenu.

Traduction rapide des verbes

Ce que l'enonce demande	Droit dossier	Droit fichier
Traverser un dossier, acceder a un nom connu	<code>--x</code>	selon l'action finale
Lister les noms du dossier	<code>r-x</code> conseille	aucun besoin sur les fichiers
Lire un fichier connu sans lister	<code>--x</code>	<code>r--</code>
Lister et lire les fichiers	<code>r-x</code>	<code>r--</code>
Modifier un fichier existant	<code>--x</code>	<code>-w-</code> ou <code>rw-</code>
Lire et modifier un fichier existant	<code>--x</code>	<code>rw-</code>
Ajouter un fichier dans un dossier	<code>-wx</code>	pas encore de fichier
Effacer un fichier d'un dossier	<code>-wx</code>	peu importe
Interdire tout aux autres	<code>---</code>	<code>---</code>

Pour `ls` simple, `r` montre les noms. En pratique d'examen, mettre souvent `r-x` sur le dossier quand on veut "lister proprement", car `ls -l` doit aussi traverser pour lire les attributs.

Recette pour resoudre

1. Mettre le proprietaire en controle total : dossiers `rwX`, fichiers `rw-`.
2. Mettre les autres utilisateurs a `---`.
3. Pour le groupe, traiter chaque dossier selon l'objectif donne.
4. Ne jamais rendre les fichiers `.txt` executables.
5. Pour les dossiers parents seulement traversables, mettre `g=x`, pas `g=rX`.
6. Pour un dossier ou le groupe peut creer/supprimer, mettre `g=wx` ou `g=rwx` selon s'il peut aussi lister.
7. Pour un dossier ou le groupe peut lire les fichiers, mettre au minimum `g=x` sur le dossier et `g=r` sur les fichiers.

Commandes modeles

Base propre, comme dans le corrigé du prof :

```
# racine traversable seulement par le groupe, rien pour les autres
chmod u=rwX,g=X,o= ./pier

# tout remettre prive dans les sous-dossiers
chmod -R u=rwX,go= ./pier/d[1-2]

# dossier lisible + traversable, fichiers lisibles
chmod -R g=rX ./pier/d1

# dossier et fichiers lisibles/modifiables par le groupe
chmod -R g=rwX ./pier/d2
```

Cas plus precis :

```
# Lire un fichier connu sans lister le dossier
chmod g=x ./secret
chmod g=r ./secret/*.txt

# Lister les noms sans lire les fichiers
chmod g=rx ./index
chmod g= ./index/*.txt

# Modifier un fichier connu sans lister ni creer
chmod g=x ./work
chmod g=rw ./work/*.txt

# Deposer et supprimer sans lister ni lire les fichiers existants
chmod g=wx ./drop
chmod g= ./drop/*.txt

# Lister, lire, modifier, ajouter et supprimer
chmod -R g=rwX ./partage
```

Comment tester comme le prof

Pour chaque objectif, il faut prouver ce qui marche et ce qui est refuse :

```
# traverser mais pas lister
cd ./pier          # autorise
ls ./pier          # refuse
touch ./pier/test.txt # refuse

# lire dans d1 mais pas modifier
ls ./pier/d1       # autorise
cat ./pier/d1/f1.txt # autorise
touch ./pier/d1/f10.txt # refuse
echo test >> ./pier/d1/f1.txt # refuse

# ajouter/supprimer dans d2
touch ./pier/d2/f10.txt # autorise
rm ./pier/d2/f10.txt    # autorise
```

Pieges specifiques a ces exercices

- Pour **supprimer** un fichier, le droit important est **w+x** sur le dossier parent.
- Pour **modifier le contenu** d'un fichier existant, il faut **x** sur le dossier et **w** sur le fichier.
- Pour **lire** un fichier, il faut **x** sur tous les dossiers du chemin et **r** sur le fichier.
- Pour **lister** un dossier proprement, mettre **r+x** sur le dossier.
- **chmod -R g=rwX dossier** donne **rw** aux fichiers et **rwX** aux dossiers, sans rendre les fichiers texte executables.

umask

Les droits de base sont **666** pour un fichier et **777** pour un repertoire. Le **umask** retire des droits.

```
umask
umask 0022      # fichiers 644, repertoires 755
umask 0077      # fichiers 600, repertoires 700
```

Bits speciaux

Bit	Exemple	Effet principal
setuid	chmod u+s prog	executable lance avec l'UID du proprietaire
SGID fichier	chmod g+s prog	executable lance avec le GID du groupe
SGID repertoire	chmod g+s par-tage	nouveaux fichiers heritent du groupe du repertoire
sticky	chmod +t /tmp	seuls proprietaire, proprietaire du dossier ou root peuvent supprimer

En octal, les bits speciaux sont le premier chiffre : **4xxx** setuid, **2xxx** SGID, **1xxx** sticky.
Exemple courant : **/tmp** en **1777**.

9. Processus et jobs

```
ps
ps -ef
ps -aux
ps -axf
top
htop

pgrep ssh
kill PID
kill -15 PID
kill -9 PID
kill -l
killall nom
```

Jobs

```
cmd &
Ctrl+Z
jobs
fg %1
bg %1
kill %1
nohup cmd &
disown
```

Variable	Sens
\$\$	PID du shell courant
#!	PID du dernier processus en arriere-plan
\$?	code de sortie de la derniere commande

10. Variables, environnement et alias

Variables shell et variables d'environnement

```
var="valeur"
echo "$var"
echo "${var}suffixe"
export var
export var="valeur"
unset var
```

Une variable non exportée existe seulement dans le shell courant. Un processus fils ne la voit pas. Une variable exportée est transmise aux processus fils.

Variables utiles

Variable	Sens
<code>\$HOME</code>	dossier personnel
<code>\$USER</code> / <code>\$LOGNAME</code>	utilisateur
<code>\$UID</code>	UID numerique
<code>\$PATH</code>	chemins de recherche
<code>\$PWD</code>	repertoire courant
<code>\$SHELL</code>	shell de login
<code>\$PS1</code>	invite
<code>\$IFS</code>	separateurs de champs

Sourcer ou executer

```
./script.sh      # nouveau processus, variables perdues a la fin
source script.sh # meme shell, variables gardees
. script.sh      # equivalent POSIX
```

Alias

```
alias ll='ls -la'
alias lt='ls -lt'
alias ..='cd ..'
alias fin='tail -2'
alias
alias lt
unalias lt
echo "alias lt='ls -lt'" >> ~/.bashrc
source ~/.bashrc
```

Un alias est une substitution textuelle faite par le shell interactif. Pour un script, preferer une fonction ou une commande explicite.

11. Scripts Bash robustes

Squelette

```
#!/bin/bash
set -e
set -u

if [ "$#" -lt 1 ]; then
    echo "Usage: $0 fichier" >&2
    exit 1
fi

fichier="$1"
echo "Traitement de $fichier"
```

`set -e` arrete sur erreur non geree. `set -u` signale les variables non definies. `set -x` sert au debug mais ne doit pas rester dans un rendu propre.

Parametres positionnels

Variable	Sens
<code>\$0</code>	nom du script
<code>\$1</code> , <code>\$2</code>	arguments
<code>\$#</code>	nombre d'arguments
<code>"\$@"</code>	tous les arguments, chacun garde comme argument separe
<code>"\$*"</code>	tous les arguments en une seule chaine
<code>shift</code>	decale les arguments

Dans un script, `"$@"` est presque toujours le bon choix pour transmettre tous les arguments.

```
for arg in "$@"; do
    echo "arg=$arg"
done
```

Tests

```
[ "$n" -eq 5 ]          # nombres: -eq -ne -lt -le -gt -ge
[ "$s" = "abc" ]       # chaines
[ -z "$s" ]            # vide
[ -n "$s" ]            # non vide
[ -f "$f" ]            # fichier ordinaire
[ -d "$d" ]            # repertoire
[ -r "$f" ]            # lisible
[ -w "$f" ]            # writable
[ -x "$f" ]            # executable/traversable
[[ "$s" == *7* ]]
[[ "$n" =~ ^[0-9]+$ ]]
(( n > 5 ))
```

Structures

```
if [ "$x" -gt 0 ]; then
    echo positif
elif [ "$x" -eq 0 ]; then
    echo zero
else
    echo negatif
fi

for f in *.txt; do
    echo "$f"
done

i=1
while [ "$i" -le 10 ]; do
    echo "$i"
    i=$((i + 1))
done

case "$1" in
    start) echo "demarrage" ;;
    stop) echo "arret" ;;
    *) echo "usage: $0 {start|stop}" >&2; exit 1 ;;
esac
```

Arithmetique, fonctions et lecture

```
i=$((i + 1))
((i++))

ma_fonction() {
    local nom="$1"
    echo "Bonjour $nom"
    return 0
}

read -p "Nom: " nom
read -s -p "Password: " pw
```

Here-documents

```
cat <<EOF
Bonjour $USER
EOF

cat <<'EOF'
Bonjour $USER
EOF

cat <<EOF > "$HOME/rapport.txt"
Rapport genere par $LOGNAME le $(date)
EOF
```

Le mot limite final (**EOF** ici) doit etre seul sur sa ligne. Avec **<<'EOF'** , les variables ne sont pas substituees.

12. Utilisateurs et groupes

Fichiers systeme

Fichier	Contenu
/etc/passwd	user:x:UID:GID:GECOS:home:shell
/etc/shadow	mots de passe hashes, root only
/etc/group	groupe:x:GID:membres
/etc/gshadow	infos securisees des groupes

Inspection fiable

```
id lucien
groups lucien
getent passwd lucien
getent group stock
```

`getent` interroge les bases configurees du systeme, pas seulement les fichiers locaux.

Creation et modification

```
sudo adduser lucien
sudo useradd -m lucien
sudo useradd -m -s /bin/bash -G users,sudo lucien
sudo passwd lucien
sudo userdel -r lucien

sudo groupadd stock
sudo groupdel stock

sudo usermod -aG stock lucien
sudo usermod -g users lucien
sudo usermod -L lucien
sudo usermod -U lucien
```

Piege majeur : `usermod -G stock lucien` remplace les groupes secondaires. Pour ajouter sans ecraser, utiliser `usermod -aG stock lucien`.

Script de gestion d'utilisateurs

Les points attendus dans un script propre : verifier root (`$UID`), verifier les arguments, verifier que `N` est numerique, utiliser des quotes, creer le groupe avec `getent group` si besoin, afficher les erreurs sur `stderr`.

PARTIE 2 - VIRTUALISATION

1. Concepts

Notion	Role
Virtualisation	execution de plusieurs systemes sur un meme hote
Hyperviseur type 1	proche du materiel, ex. KVM
Hyperviseur type 2	au-dessus d'un OS hote, ex. VirtualBox
KVM	module noyau Linux fournissant la virtualisation materielle
QEMU	emulation et modelisation du materiel virtuel
libvirt	couche d'administration commune
virsh	CLI de libvirt
virt-manager	interface graphique de libvirt
virt-install	creation de VM en ligne de commande

VM et conteneur

Une VM possede son propre noyau invite et une isolation plus forte. Un conteneur partage le noyau de l'hote et demarre plus vite, avec une empreinte plus faible. Pour l'evaluation L12, le coeur pratique est KVM/libvirt/LVM/reseau.

URI libvirt

```
qemu:///system
qemu:///session
qemu+ssh://user@host/system
```

En cours, les commandes utilisent surtout `qemu:///system`.

2. Installation, verification et acces libvirt

```
which virsh
which virt-manager
id
groups
dpkg -l | grep -E 'libvirt|qemu|kvm'
```

L'utilisateur doit faire partie du groupe `libvirt` pour administrer plus confortablement les VM.

```
sudo usermod -aG libvirt "$USER"
```

Apres modification de groupe, il faut rouvrir la session ou lancer un nouveau shell de login.

3. VM de reference pour l'evaluation

La lecon 11 indique deux VM de reference a installer avant l'evaluation.

```
sudo cp ./kvmRef1.qcow2 /var/lib/libvirt/images
virsh -c qemu:///system define ./kvmRef1.xml

sudo cp ./kvmRef2.img /var/lib/libvirt/images
virsh -c qemu:///system define ./kvmRef2.xml
```

Verification attendue :

```
virsh -c qemu:///system list --all
virsh -c qemu:///system vol-list default
virsh -c qemu:///system start kvmRef1
virsh -c qemu:///system start kvmRef2
virsh -c qemu:///system domifaddr kvmRef1
virsh -c qemu:///system domifaddr kvmRef2
virsh -c qemu:///system console kvmRef1
ssh -o IdentitiesOnly=yes debian@IP_DE_LA_VM
```

Points a retenir : **define** enregistre une VM depuis son XML; copier le disque ne suffit pas. **domifaddr** donne l'IP si la VM repond via l'agent ou les baux reseau connus. **Ctrl+]** quitte **virsh console**.

4. Administration des VM avec virsh

Lister et inspecter

```
virsh -c qemu:///system list
virsh -c qemu:///system list --all
virsh -c qemu:///system list --all --name
virsh -c qemu:///system dominfo vm1
virsh -c qemu:///system domifaddr vm1
virsh -c qemu:///system domblklist vm1
virsh -c qemu:///system domiflist vm1
virsh -c qemu:///system dumpxml vm1
virsh -c qemu:///system dumpxml vm1 > vm1.xml
```

Cycle de vie

```
virsh -c qemu:///system start vm1
virsh -c qemu:///system shutdown vm1
virsh -c qemu:///system destroy vm1
virsh -c qemu:///system reboot vm1
virsh -c qemu:///system suspend vm1
virsh -c qemu:///system resume vm1
virsh -c qemu:///system autostart vm1
virsh -c qemu:///system autostart --disable vm1
```

shutdown demande un arret propre via ACPI. **destroy** force l'arret, comme si on retirait l'alimentation.

Definition et suppression

```
virsh -c qemu:///system define vm1.xml
virsh -c qemu:///system undefine vm1
virsh -c qemu:///system undefine vm1 --remove-all-storage
virsh -c qemu:///system edit vm1
```

undefine retire la definition; les disques restent presents sauf option explicite de suppression.

5. Creation de VM avec virt-install

Creation en console serie

```
virt-install --connect qemu:///system \  
  --name vm1 \  
  --memory 1024 \  
  --vcpus 1 \  
  --disk path=/var/lib/libvirt/images/vm1.img,format=raw,bus=virtio,size=6 \  
  --location ~/iso/debian-13.3.0-amd64-netinst.iso \  
  --network network=default,model=virtio \  
  --osinfo name=debian11 \  
  --graphics none \  
  --console pty,target_type=serial \  
  --extra-args "console=ttyS0,115200n8"
```

Options importantes

Option	Role
--name	nom de la VM
--memory	RAM en Mio
--vcpus	nombre de CPU virtuels
--disk	disque, format, bus et taille
--cdrom	installation via ISO graphique
--location	installation permettant arguments kernel
--network network=default,model=virtio	branchement reseau
--graphics none	pas d'affichage graphique
--console pty,target_type=serial	console serie
--extra-args "console=ttyS0..."	sortie installateur vers serie
--print-xml	genere le XML sans creer la VM

Variante graphique ou import

```
virt-install --name vm2 --memory 2048 --vcpus 2 \  
  --disk size=10 --cdrom ~/iso/debian.iso \  
  --network network=default --graphics spice  
  
virt-install --name vm3 --memory 1024 --vcpus 1 \  
  --disk path=/var/lib/libvirt/images/vm3.qcow2 \  
  --network network=default --import --osinfo name=debian11
```

6. Stockage, formats, pools et volumes

Formats disque

Format	Avantages	Limites
raw	simple, rapide, taille fixe	pas de snapshot interne
qcow2	allocation dynamique, snapshots	leger surcout
LV LVM	performant, snapshots hote	plus complexe

qemu-img

```
qemu-img create -f raw vm.raw 6G
qemu-img create -f qcow2 vm.qcow2 6G
qemu-img info vm.qcow2
qemu-img convert -f raw -O qcow2 vm.raw vm.qcow2
qemu-img resize vm.qcow2 +2G
```

resize agrandit le disque virtuel, pas automatiquement la partition ni le système de fichiers.

Pools libvirt

```
virsh -c qemu:///system pool-list --all
virsh -c qemu:///system pool-dumpxml default
virsh -c qemu:///system pool-define-as vms-home dir --target /home/user/vms
virsh -c qemu:///system pool-start vms-home
virsh -c qemu:///system pool-autostart vms-home

virsh -c qemu:///system vol-list default
virsh -c qemu:///system vol-create-as default vm.img 4G --format raw
```

Attacher un disque

```
sudo qemu-img create -f raw /var/lib/libvirt/images/vol2.img 2G
sudo chown libvirt-qemu:kvm /var/lib/libvirt/images/vol2.img

virsh -c qemu:///system attach-disk vm1 \
  /var/lib/libvirt/images/vol2.img vdb \
  --driver qemu --subdriver raw \
  --config --live

virsh -c qemu:///system domblklist vm1
```

--live applique à chaud. **--config** rend le changement persistant.

7. Clonage et post-installation

Clonage fichier

```
virt-clone --connect qemu:///system \
  --original vm1 \
  --name vm1b \
  --file /var/lib/libvirt/images/vm1b.img
```

virt-clone copie le disque, change le nom, l'UUID, les MAC et les chemins disque dans le XML.

Clonage vers LV LVM

```
sudo lvcreate -L 6G -n kvm-vm3 vg
sudo dd if=/var/lib/libvirt/images/vm1.raw \
  of=/dev/vg/kvm-vm3 \
  bs=4M status=progress

virt-clone --connect qemu:///system \
  --original vm1 \
  --name vm3 \
  --file /dev/vg/kvm-vm3 \
  --preserve-data
```

--preserve-data est essentiel après un **dd** vers un LV déjà rempli. Sans cette option, **virt-clone** peut réécrire la cible.

Post-clonage

Dans la VM clonee :

```
sudo hostnamectl set-hostname vm3
sudo sed -i 's/vm1/vm3/g' /etc/hosts
sudo rm /etc/ssh/ssh_host_*
sudo dpkg-reconfigure openssh-server
sudo systemd-machine-id-setup
```

Raison : eviter deux machines avec le meme hostname, les memes cles SSH serveur ou la meme identite machine.

8. Console serie

Pour que `virsh console vm1` soit utilisable, l'invite de login doit sortir sur `ttys0`.

Dans la VM :

```
sudo nano /etc/default/grub
```

Valeurs typiques :

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet console=ttys0,115200n8"
GRUB_TERMINAL=serial
GRUB_SERIAL_COMMAND="serial --speed=115200"
```

Puis :

```
sudo update-grub
sudo reboot
virsh -c qemu:///system console vm1
```

Quitter la console : `Ctrl+]`.

9. LVM, extension et snapshots

Notions

Element	Role
PV	disque ou partition donne a LVM
VG	groupe de volumes regroupant des PV
LV	volume logique utilise comme disque/partition

```
sudo pvs
sudo vgs
sudo lvs
sudo pvdisplay
sudo vgdisplay
sudo lvdisplay
```

Snapshot LVM

```
virsh -c qemu:///system destroy vm3
sudo lvcreate -s /dev/vg/kvm-vm3 -L 1G -n kvm-vm3-s1
sudo lvs
```

Revenir a l'etat du snapshot :

```
virsh -c qemu:///system destroy vm3
sudo lvconvert --merge /dev/vg/kvm-vm3-s1
virsh -c qemu:///system start vm3
```

Jeter le snapshot et garder l'etat courant :

```
sudo lvremove /dev/vg/kvm-vm3-s1
```

La taille du snapshot doit absorber les changements ecrits depuis sa creation.

Etendre une VM sans LVM interne

Cote hote :

```
virsh -c qemu:///system destroy vm3
sudo lvextend -L +2G /dev/vg/kvm-vm3
sudo kpartx -av /dev/vg/kvm-vm3
sudo e2fsck -fy /dev/mapper/vg-kvm--vm3p1
```

Ensuite, agrandir la partition avec `fdisk` ou un outil adapte, relire la table, puis :

```
sudo resize2fs /dev/mapper/vg-kvm--vm3p1
sudo kpartx -d /dev/vg/kvm-vm3
virsh -c qemu:///system start vm3
```

Etendre une VM avec LVM interne

Cote hote :

```
virsh -c qemu:///system destroy vm4
sudo lvextend -L +2G /dev/vg/kvm-vm4
virsh -c qemu:///system start vm4
```

Dans la VM :

```
sudo fdisk /dev/vda          # creer vda3 de type 8e
sudo partx -u /dev/vda
sudo pvcreate /dev/vda3
sudo vgextend vm-vg /dev/vda3
sudo lvextend -l +100%FREE /dev/vm-vg/root
sudo resize2fs /dev/vm-vg/root
df -h /
```

10. Reseaux virtuels libvirt

Reseau default

Le reseau `default` est en NAT, generalement sur `virbr0`, avec la plage `192.168.122.0/24`. La passerelle est souvent `192.168.122.1` et le DHCP est fourni par libvirt.

```
virsh -c qemu:///system net-list --all
virsh -c qemu:///system net-dumpxml default
virsh -c qemu:///system net-edit default
```

Reseau isole

Un reseau isole ne contient pas de `<forward>`. Les VM communiquent entre elles, mais ne sortent pas automatiquement vers l'exterieur.

```
<network>
  <name>net-isole</name>
  <bridge name='virbr10' stp='on' delay='0' />
  <ip address='10.10.10.1' netmask='255.255.255.0'>
    <dhcp>
      <range start='10.10.10.100' end='10.10.10.200' />
    </dhcp>
  </ip>
</network>
```

```
virsh -c qemu:///system net-define net-isole.xml
virsh -c qemu:///system net-start net-isole
virsh -c qemu:///system net-autostart net-isole
virsh -c qemu:///system net-destroy net-isole
```

Reservation DHCP par MAC

```
<host mac='52:54:00:19:85:3e' name='vm3' ip='192.168.122.254' />
```

La reservation se place dans le bloc `<dhcp>`. Il est plus propre de choisir une IP hors de la plage dynamique, ou d'ajuster la plage.

11. Topologie vm1, vm2, vm3 et passerelle

Topologie vue dans les activites 0700/0710 :

```
default / virbr0    192.168.122.0/24
net-isole          10.10.10.0/24

vm1  -> default uniquement
vm2  -> net-isole uniquement
vm3  -> default + net-isole
```

Adresses de reference souvent utilisees :

VM	Interface	IP	Role
vm1	default	192.168.122.x	client cote default
vm2	net-isole	10.10.10.x	client isole
vm3	default	192.168.122.254	relais cote default
vm3	net-isole	10.10.10.2	relais cote isole

Verification reseau

```
ip -br a
ip route
virsh -c qemu:///system domiflist vm3
virsh -c qemu:///system domifaddr vm3
ping -c 4 192.168.122.254
ping -c 4 10.10.10.2
```

Forwarding IPv4

Sur `vm3`, temporaire :

```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
sudo sysctl -w net.ipv4.ip_forward=1
cat /proc/sys/net/ipv4/ip_forward
```

Permanent :

```
sudo nano /etc/sysctl.conf
```

Ajouter ou commenter :

```
net.ipv4.ip_forward=1
```

Appliquer :

```
sudo sysctl -p
```

Routes pour faire passer vm2 vers default via vm3

Sur **vm2** :

```
sudo ip route add 192.168.122.0/24 via 10.10.10.2 dev ens3
ip route
```

Sur **vm1** , route de retour :

```
sudo ip route add 10.10.10.0/24 via 192.168.122.254 dev ens3
ip route
```

Sans route de retour, le paquet peut partir de **vm2** , atteindre **vm1** , puis la reponse se perdre.

Routes persistantes avec `/etc/network/interfaces`

Sur **vm2** :

```
allow-hotplug ens3
iface ens3 inet dhcp
    post-up ip route add 192.168.122.0/24 via 10.10.10.2 dev ens3 || true
    pre-down ip route del 192.168.122.0/24 via 10.10.10.2 dev ens3 || true
```

Sur **vm1** :

```
allow-hotplug ens3
iface ens3 inet dhcp
    post-up ip route add 10.10.10.0/24 via 192.168.122.254 dev ens3 || true
    pre-down ip route del 10.10.10.0/24 via 192.168.122.254 dev ens3 || true
```

12. Configuration reseau dans une VM

`/etc/network/interfaces`

```
source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback

allow-hotplug ens3
iface ens3 inet dhcp
```

Pour une VM avec deux cartes :

```
allow-hotplug ens3
iface ens3 inet dhcp

allow-hotplug ens9
iface ens9 inet dhcp
```


Netplan

```
network:
  version: 2
  ethernets:
    all-en:
      match:
        name: en*
      dhcp4: true
      routes:
        - to: 10.10.10.0/24
          via: 192.168.122.254
```

```
sudo netplan apply
sudo netplan status
```

Inspection des services reseau

```
ip a
ip -br a
ip link
ip route
ss -tulpn
ping -c4 host
traceroute host
```

13. SSH, cles, jump host et tunnel

Connexion simple

```
ssh debian@192.168.122.254
ssh -p 2222 debian@host
ssh -i ~/.ssh/id_ed25519 debian@host
```

Cles SSH

```
ssh-keygen -t ed25519
ssh-copy-id debian@192.168.122.254
```

Installation manuelle de la cle publique :

```
cat ~/.ssh/id_ed25519.pub | ssh debian@host \
'mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 700 ~/.ssh && chmod 600 ~/.ssh/
authorized_keys'
```

La cle privee doit rester protegee, typiquement `chmod 600 ~/.ssh/id_ed25519`.

Jump host

Depuis `vm1`, atteindre `vm2` via `vm3` :

```
ssh -J debian@192.168.122.254 debian@10.10.10.168
```

Lecture : se connecter d'abord a `vm3`, puis ouvrir depuis `vm3` une connexion vers `vm2`.

Equivalent dans `~/.ssh/config` :

```
Host vm2-isole
  HostName 10.10.10.168
  User debian
  ProxyJump debian@192.168.122.254
```

Tunnel local

Depuis **vm1** :

```
ssh -N -L 2222:10.10.10.168:22 debian@192.168.122.254
```

Dans un autre terminal :

```
ssh -p 2222 debian@localhost
```

-N ne lance pas de shell distant. **-L** cree une redirection locale : le port local **2222** mene au port **22** de **vm2** en passant par **vm3**.

Forme a retenir : **ssh -L port_local:hote_final:port_final user@machine_relais**.

Tunnel inverse

```
ssh -N -R 9000:localhost:80 user@host
```

Le port **9000** cote serveur distant redirige vers le port **80** local.

14. Post-installation et controles finaux

Hostname et **/etc/hosts**

```
sudo hostnamectl set-hostname vm1
echo "vm1" | sudo tee /etc/hostname
sudo sed -i "s/127.0.1.1.*/127.0.1.1\tvm1/" /etc/hosts
hostname
```

Regeneration des cles SSH serveur

```
sudo rm /etc/ssh/ssh_host_*
sudo dpkg-reconfigure openssh-server
for f in /etc/ssh/ssh_host_*.pub; do
    ssh-keygen -lf "$f"
done
```

Paquets et acces

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y openssh-server sudo vim
systemctl status ssh
```

Reflexes de verification avant test

```
virsh -c qemu:///system list --all
virsh -c qemu:///system net-list --all
virsh -c qemu:///system vol-list default
virsh -c qemu:///system domifaddr vm1
virsh -c qemu:///system domblklist vm1
virsh -c qemu:///system domiflist vm1
ip -br a
ip route
ssh debian@IP
```

Verifier chaque couche dans l'ordre : VM definie, disque present, VM demarree, interface branchee au bon reseau, IP recue, route correcte, service SSH actif, puis connexion.

ANNEXES

Cheatsheet 1 page - commandes probables au test

Shell - commandes rapides

```
# Aide et resolution de commandes
man passwd
apropos password
help cd
type cd
type ls
command -v ls

# Navigation et fichiers
pwd
ls -la
mkdir -p a/b/c
touch data_{01..15}.txt
cp source dest
cp -r dossier copie
mv ancien nouveau
rm fichier
ln cible lien_hard
ln -s cible lien_symbolique

# Redirections et pipes
cmd > out.txt
cmd >> out.txt
cmd 2> err.txt
cmd > out.txt 2>&1
cmd 2>/dev/null
echo "Erreur" >&2
cut -d: -f1 /etc/passwd | sort | uniq
ls -l | tee listing.log | grep '\.txt$'

# Filtres
wc -l /etc/passwd
cut -d: -f1,3 /etc/passwd
awk -F: '$3 >= 1000 && $3 <= 1999 {print $1}' /etc/passwd
w | tr -s ' ' ':' | cut -d: -f1,6
sort fichier | uniq -c
grep -E '^[a-z]+$' fichier
grep '\.txt$' listing.txt
```

find

```
find . -name '*.txt'
find . -type f
find . -type d
find . -size +1M
find . -mtime -7
find . -user lucien
find . -group users
find . -perm 644
find . -perm -111
find . -name '*.tmp' -delete
find . -name '*.sh' -exec chmod +x {} \;
find /etc -name '*.conf' -exec cp {} ~/backup_conf/ \; 2>/dev/null
```

Permissions

Octal	Litteral	Usage typique
600	rw-----	cle SSH privee
644	rw-r--r--	fichier texte standard
700	rwX-----	~/.ssh , dossier prive
755	rwXr-Xr-X	dossier ou executable public
1777	rwXrwxrwt	/tmp , sticky bit
2750	rwXr-s---	dossier partage avec SGID

```
chmod u+x script.sh
chmod u=rwx,g=rx,o= dossier
chmod 644 fichier
chmod 755 dossier
chmod 600 ~/.ssh/id_ed25519
chmod 700 ~/.ssh
chmod g+s partage
chmod +t dossier_public
chown user:group fichier
chgrp group fichier
umask 0022
```

Scripts Bash

```
#!/bin/bash
set -e
set -u

if [ "$#" -ne 2 ]; then
    echo "Usage: $0 {create|delete} N" >&2
    exit 1
fi

action="$1"
n="$2"

if ! [[ "$n" =~ ^[0-9]+$ ]]; then
    echo "Erreur: N doit etre un entier" >&2
    exit 1
fi

for arg in "$@"; do
    echo "$arg"
done
```

Utilisateurs et groupes

```
id lucien
groups lucien
getent passwd lucien
getent group stock

sudo useradd -m -s /bin/bash lucien
sudo useradd -m -g exam_group exam_user1
sudo userdel -r lucien
sudo passwd lucien
sudo groupadd stock
sudo usermod -aG stock lucien
sudo usermod -g users lucien
sudo chage -l lucien
```

Virtualisation - virsh

```
virsh -c qemu:///system list --all
virsh -c qemu:///system list --all --name
virsh -c qemu:///system start vm1
virsh -c qemu:///system shutdown vm1
virsh -c qemu:///system destroy vm1
virsh -c qemu:///system dominfo vm1
virsh -c qemu:///system domifaddr vm1
virsh -c qemu:///system domblklist vm1
virsh -c qemu:///system domiflist vm1
virsh -c qemu:///system dumpxml vm1 > vm1.xml
virsh -c qemu:///system define vm1.xml
virsh -c qemu:///system undefine vm1
virsh -c qemu:///system console vm1
```

Virtualisation - creation, stockage, clonage

```
qemu-img create -f raw vm.raw 6G
qemu-img create -f qcow2 vm.qcow2 6G
qemu-img info vm.qcow2
qemu-img resize vm.qcow2 +2G

virt-install --connect qemu:///system \
  --name vm1 \
  --memory 1024 \
  --vcpus 1 \
  --disk path=/var/lib/libvirt/images/vm1.img,format=raw,bus=virtio,size=6 \
  --location ~/iso/debian.iso \
  --network network=default,model=virtio \
  --graphics none \
  --console pty,target_type=serial \
  --extra-args "console=ttyS0,115200n8"

virt-clone --connect qemu:///system \
  --original vm1 \
  --name vm1b \
  --file /var/lib/libvirt/images/vm1b.img

sudo lvcreate -L 6G -n kvm-vm3 vg
sudo dd if=/var/lib/libvirt/images/vm1.raw of=/dev/vg/kvm-vm3 bs=4M status=progress
virt-clone --connect qemu:///system --original vm1 --name vm3 \
  --file /dev/vg/kvm-vm3 --preserve-data
```

LVM, reseau et SSH

```
# LVM
sudo pvs
sudo vgs
sudo lvs
sudo lvextend -L +2G /dev/vg/kvm-vm3
sudo lvcreate -s /dev/vg/kvm-vm3 -L 1G -n kvm-vm3-s1
sudo lvconvert --merge /dev/vg/kvm-vm3-s1

# Reseaux libvirt
virsh -c qemu:///system net-list --all
virsh -c qemu:///system net-dumpxml default
virsh -c qemu:///system net-edit default
virsh -c qemu:///system net-define net-isole.xml
virsh -c qemu:///system net-start net-isole
virsh -c qemu:///system net-autostart net-isole

# Reseau dans une VM
ip -br a
ip route
sudo sysctl -w net.ipv4.ip_forward=1
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
sudo ip route add 192.168.122.0/24 via 10.10.10.2 dev ens3
sudo ip route add 10.10.10.0/24 via 192.168.122.254 dev ens3

# SSH
ssh debian@192.168.122.254
ssh-keygen -t ed25519
ssh-copy-id debian@192.168.122.254
ssh -J debian@192.168.122.254 debian@10.10.10.168
ssh -N -L 2222:10.10.10.168:22 debian@192.168.122.254
ssh -p 2222 debian@localhost
```

Pieges frequents au test

Shell

1. `find . -name *.h` est dangereux : le shell peut developper `*.h` avant `find`. Utiliser `find . -name '*.h'`.
2. `grep ".txt"` ne cherche pas un vrai point : `.` est une regex. Utiliser `grep '\.txt$'` ou `grep -F '.txt'`.
3. `>` ecrase un fichier, `>>` ajoute a la fin.
4. `2>/dev/null` masque les erreurs mais ne transforme pas forcement l'echec en succes.
5. `$?` contient le code de sortie de la derniere commande seulement.
6. Sans quotes, `$var` subit word splitting et globbing. Utiliser presque toujours `"$var"`.
7. Pour transmettre tous les arguments d'un script, preferer `"$@"` a `*$*`.
8. `./script.sh` lance un sous-shell; `source script.sh` modifie le shell courant.
9. Une variable non exportee n'est pas visible par les processus fils.
10. `echo "Erreur" >&2` envoie le message sur `stderr`.

Permissions

1. Sur un repertoire, `x` est indispensable pour traverser et resoudre les noms.
2. `w` sans `x` sur un repertoire est presque inutile.
3. Supprimer un fichier depend du droit `w+x` sur le repertoire parent, pas du droit `w` sur le fichier.
4. `cp` a besoin de lire le fichier source et d'ecrire dans le dossier destination.
5. `mv` entre dossiers a besoin de `w+x` dans le dossier source et destination.

6. `chmod 600 dossier` donne `drw-----`, ce qui casse l'accès car il manque `x`.
7. `umask` retire des droits : `0022` donne fichiers `644`, dossiers `755`.
8. `SGID` sur un répertoire force l'héritage du groupe du répertoire.
9. `sticky` évite que n'importe qui supprime les fichiers des autres dans un dossier ouvert.
10. `usermod -G groupe user` remplace les groupes secondaires; utiliser `usermod -aG groupe user`.

Scripts Bash

1. Les espaces dans `[ "$i" -le "$n" ]` sont obligatoires.
2. Il faut `do` dans une boucle `for` ou `while`.
3. Vérifier les arguments avant d'utiliser `$1`, surtout avec `set -u`.
4. Les erreurs d'usage doivent aller sur `stderr` avec `>&2`.
5. Un script de gestion d'utilisateurs doit vérifier `$UID -eq 0`.
6. Tester un entier avec `[[ "$n" =~ ^[0-9]+$ ]]` avant une comparaison numérique.
7. `local` ne s'utilise que dans une fonction.
8. Le mot de fin d'un here-document doit être seul sur sa ligne.

Virtualisation

1. Copier un disque de VM ne suffit pas : il faut aussi définir la VM avec `virsh define` ou `virt-clone`.
2. `shutdown` est propre; `destroy` est un arrêt forcé.
3. `undefine` retire la définition mais ne supprime pas forcément le disque.
4. `virsh console` reste muet si GRUB et le noyau ne sortent pas sur `ttyS0`.
5. Après clonage, changer `hostname`, `/etc/hosts`, clés SSH hôte et éventuellement `machine-id`.
6. Pour cloner vers un LV déjà rempli par `dd`, `--preserve-data` est indispensable.
7. `qemu-img resize` ou `lvextend` agrandit le disque; il faut encore agrandir partition/LVM interne/filesystem.
8. Un snapshot LVM doit être créé VM arrêtée pour éviter un état disque incohérent.
9. Un réseau libvirt isole n'a pas de `<forward>` : pas de sortie automatique.
10. Une réservation DHCP doit utiliser la bonne MAC de la bonne interface.
11. Pour faire routeur entre `default` et `net-isole`, il faut `ip_forward=1` sur la passerelle.
12. Sans route de retour sur `vm1`, `vm2` peut envoyer un paquet mais ne pas recevoir la réponse.
13. `ssh -J` fait un saut via une machine relais; `ssh -L` crée un tunnel local.
14. `-N` avec SSH garde le tunnel ouvert sans lancer de shell distant.
15. Vérifier dans l'ordre : VM démarrée, interface branchée, IP, route, service SSH, connexion.